

Heat in a Cold Apartment

Solving the Laplace equation by Dirichlet–Neumann domain decomposition with MPI

FMNN25 / NUMN21 — Advanced Numerical Algorithms with Python/SciPy — Project 1 and Project 1a

This document explains, in plain terms, what the project asks, how the method works, how the code implements it, and what the answers turned out to be. It is written to be read before a presentation: every formula is explained in words, every non-obvious line of code is unpacked, and the last section is a list of the questions most likely to be asked, with answers.

Contents

1. The problem
2. The physics, in one sentence
3. From a physical law to a system of equations
4. Why the apartment is split into rooms
5. The Dirichlet–Neumann handshake
6. The matrices (Task 1)
7. Relaxation
8. How the code is organised
9. The MPI layer
10. Results for Project 1 (Tasks 2, 3, 4)
11. Project 1a: the 2.5-room apartment
12. Was the code extendible?
13. Task 2 of Project 1a: computing at $\Delta x = 1/100$
14. Summary of all results
15. Presentation crib sheet

1. The problem

We are buying a flat and want to know whether it will be warm in winter. The flat is modelled as a two-dimensional region made of rectangular rooms, and we want the **steady** temperature everywhere — the temperature the flat settles at after the heating has been running long enough that nothing changes any more.

The walls carry three kinds of fixed temperature:

Boundary	Symbol	Temperature	What it is
Normal wall	$\partial\Omega \setminus (\Gamma_H \cup \Gamma_{WF})$	15 °C	An ordinary wall shared with a neighbouring flat
Heater wall	Γ_H	40 °C	A wall with a radiator on it
Window wall	Γ_{WF}	5 °C	The big cold window

The interior temperature $u(x)$ satisfies the **Laplace equation**

$$\Delta u(x) = 0, \quad x \in \Omega \subset \mathbb{R}^2.$$

The task is to solve this on the flat, decide whether the heating is adequate, and plot the result — using MPI, with each room handled by its own process, coupled by the Dirichlet–Neumann method.

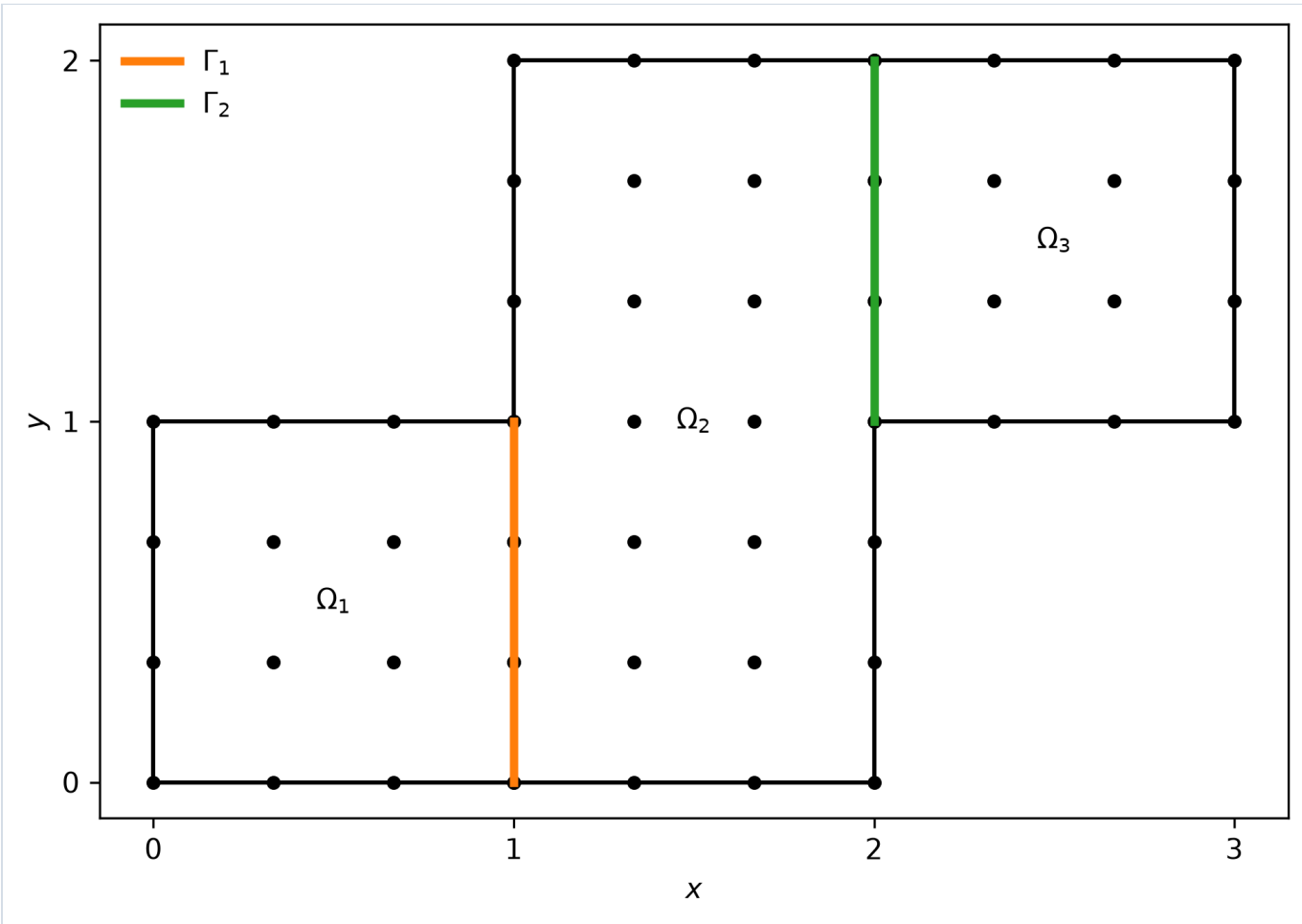


Figure 1. The 2-room flat with the grid drawn at $\Delta x = 1/3$. Ω_1 and Ω_3 are 1×1 ; Ω_2 is 1×2 . The two coloured lines Γ_1 and Γ_2 are the artificial interfaces between rooms — they are not walls, they are where we chose to cut the flat up.

2. The physics, in one sentence

Heat spreads until it settles. Once it has settled, and there is no heat source *inside* the room (here the heaters are painted on the walls, not standing in the middle of the floor), **every point ends up at the average of the points immediately around it**. A point cannot be hotter than its surroundings unless something is actively heating it.

That is the entire content of $\Delta u = 0$. Written on a grid with spacing $h = \Delta x$:

$$u_{i,j} = \frac{u_{i-1,j} + u_{i+1,j} + u_{i,j-1} + u_{i,j+1}}{4}$$

The project prescribes the standard second-order central difference approximation

$$\Delta u(x_{i,j}) \approx \frac{u_{i,j+1} + u_{i,j-1} - 4u_{i,j} + u_{i+1,j} + u_{i-1,j}}{\Delta x^2},$$

which is the same statement: setting the numerator to zero says *the node equals the average of its four neighbours*. "Second order" means the error of this approximation shrinks like h^2 — halving the mesh width quarters the error.

3. From a physical law to a system of equations

Known and unknown nodes

Lay a square grid of spacing h over the flat. Every grid node is one of two things:

- **Known** — it sits on a wall, so we are told its value: 15, 40 or 5 °C.
- **Unknown** — it is inside, and we have to compute it.

Each unknown node contributes one equation. Rearranging the averaging rule so that everything unknown is on the left:

$$4u_{i,j} - u_{i-1,j} - u_{i+1,j} - u_{i,j-1} - u_{i,j+1} = 0.$$

With N unknowns we get N equations, which is a linear system

$$A v = b.$$

- v is the list of unknown temperatures, flattened from the two-dimensional grid into one long column vector.
- A holds the coefficients: a 4 on the diagonal (the node itself) and a -1 for every neighbour that is *also unknown*.
- b collects the known wall temperatures. A wall node has no column in A , so whenever a neighbour is a wall its value is moved to the right-hand side.

Where did h go?

Multiplying $\Delta u = 0$ by $-h^2$ cancels the denominator entirely. That is why the mesh width never appears in the matrix — the coefficients are just 4 and -1 whatever the resolution. It survives in exactly one place: the flux term of the Neumann rows, $h q$, in Section 6.

Flattening: from (i, j) to a single index

A matrix row is identified by one number, but a grid node has two coordinates, so we need a numbering rule. The code uses **row-major** order: walk left to right along the bottom row of unknowns, then the next row up, and so on.

```
def interior_index(i, j, nx):  
    """Map an interior grid node to its row-major vector index."""  
    return (j - 1) * (nx - 1) + i - 1
```

The -1 s are because node $(1, 1)$ is the first unknown (index 0); row j starts after $(j - 1)$ complete rows of $(nx - 1)$ unknowns each. This function is pure bookkeeping, but every matrix entry goes through it, so it is worth being clear about.

Array convention

In the code, grids are NumPy arrays indexed `u[row, column]` where **rows increase with y** and **columns increase with x** . So `u[0, :]` is the bottom edge and `u[-1, :]` is the top edge. This is why, in the middle room, the window appears as `u[0, :] = window` and the heater as `u[-1, :] = heater`.

Solving one room

Once A and b are built, one call to `scipy.linalg.solve` gives the answer. This is a **direct** solver (Gaussian elimination / LU factorisation), not an iterative one. Worth being explicit about in a presentation:

There are two completely different things going on

Inside a single room the problem is solved **exactly**, in one shot, with no iteration. The iteration in this project is *only* about making the three rooms agree with each other at the interfaces. If the flat were a single rectangle there would be no iteration at all.

4. Why the apartment is split into rooms

Two reasons, and it is worth giving both:

1. **Shape.** The flat is an L-shape. A single Cartesian grid over an L-shape is awkward; three rectangles are trivial to grid.
2. **Parallelism.** This is a course on parallel numerics. One room per MPI process is the whole point of the exercise.

But splitting creates a new difficulty. Γ_1 and Γ_2 are **invented boundaries** — open doorways, not walls. To solve a room you need a boundary condition on every edge, and nobody told us the temperature in a doorway. That missing information is what the Dirichlet–Neumann iteration manufactures.

5. The Dirichlet–Neumann handshake

For three separately computed solutions to glue together into one physically sensible answer, two things must match at each interface:

1. The **temperature** must agree from both sides — no sudden jump.
2. The **heat flow** through the interface must agree — whatever leaves Ω_2 must be exactly what enters Ω_1 .

You cannot impose both conditions on both sides; that would be over-determined. So the two requirements are split between the neighbours:

Room	Gets told	Condition type	Computes and hands back
Ω_2 (middle)	the interface <i>temperature</i>	Dirichlet	the heat <i>flux</i> through the interface
Ω_1, Ω_3 (outer)	the interface <i>flux</i>	Neumann	the interface <i>temperature</i>

A **Dirichlet** condition prescribes the value of the solution on a boundary ("this edge is at 15 °C"). A **Neumann** condition prescribes its normal derivative ("heat is crossing this edge at this rate"). The normal derivative is the slope of the temperature as you step across the boundary, and slope is exactly what drives heat flow, so "Neumann condition" and "prescribed heat flow" mean the same thing here.

The iteration

Given u_1^k, u_2^k, u_3^k , one sweep is:

1. Solve Ω_2 with Dirichlet data on Γ_1 and Γ_2 taken from u_1^k and u_3^k . This gives u_2^{k+1} .
2. Solve Ω_1 and Ω_3 with Neumann data on Γ_1 and Γ_2 taken from u_2^{k+1} . This gives u_1^{k+1} and u_3^{k+1} .
3. Relax: $u^{k+1} \leftarrow \omega u^{k+1} + (1 - \omega)u^k$.

Start with every interface guessed at 15 °C. The project prescribes ten sweeps with $\omega = 0.8$ and $\Delta x = 1/20$.

The plain-language version

Think of Γ_1 as a doorway. We tell Ω_2 "hold the doorway at 15 °C" and it solves. From its answer we can measure how steeply the temperature drops across the doorway — that slope is the heat streaming through. We tell Ω_1 "this much heat is coming at you", and it solves, and its answer says what temperature the doorway actually settles at on its side. We hand that temperature back to Ω_2 and go round again. After a few rounds the two sides stop disagreeing.

How the flux is actually computed

```
def interface_fluxes(u2, h):
    """Compute the fluxes from the middle room into the two outer rooms."""
    n = u2.shape[1] - 1
    flux1 = (u2[1:n, 1] - u2[1:n, 0]) / h
    flux3 = (u2[n+1:-1, -2] - u2[n+1:-1, -1]) / h
    return flux1, flux3
```

Read `flux1` in words: (*temperature one grid step inside the room*) minus (*temperature on the interface*), divided by the step size. That is a one-sided finite difference approximation of the normal derivative $\partial u / \partial n$. The slicing `[1:n]` selects the interior nodes of the lower half of the left edge, which is where Ω_1 is attached; `[n+1:-1]` selects the upper half of the right edge, where Ω_3 is attached.

6. The matrices (Task 1)

Task 1 asks for the matrices at $\Delta x = 1/3$. There are two kinds of row.

Interior rows

For a node with all four neighbours available, multiplying the difference formula by $-h^2$:

$$4u_{i,j} - u_{i-1,j} - u_{i+1,j} - u_{i,j-1} - u_{i,j+1} = 0.$$

Diagonal 4, and -1 for each unknown neighbour. If a neighbour is a Dirichlet node with known value T , its -1 cannot go in the matrix, so $+T$ goes into b instead. In the code the filter `if 1 <= ni < nx and 1 <= nj < ny` is precisely the test "is this neighbour also an unknown?":

```
for j in range(1, ny):
    for i in range(1, nx):
        row = interior_index(i, j, nx)
        rows.append(row); columns.append(row); values.append(4.0)

        for ni, nj in ((i-1, j), (i+1, j), (i, j-1), (i, j+1)):
            if 1 <= ni < nx and 1 <= nj < ny:      # neighbour is also unknown
                rows.append(row)
                columns.append(interior_index(ni, nj, nx))
                values.append(-1.0)
```

Neumann interface rows: where the 3 comes from

This is the one derivation worth having at your fingertips, because it is the part of the matrix that looks strange.

In the middle room the interface nodes are *known* (we are told their temperature), so they are not unknowns at all. In an outer room it is the opposite: the interface temperature is exactly what we are solving for, so **the interface nodes are unknowns**. But such a node sits on the edge of its room and has no neighbour on the far side — only three neighbours exist.

Balancing the flux into the node on the right edge of Ω_1 , with the prescribed flux $q_{1,j}$ supplying the missing direction:

$$\frac{1}{h} \left(\frac{u_{N,j+1} - u_{N,j}}{h} - \frac{u_{N,j} - u_{N,j-1}}{h} + q_{1,j} - \frac{u_{N,j} - u_{N-1,j}}{h} \right) = 0.$$

Multiplying by $-h^2$:

$$3u_{N,j} - u_{N,j+1} - u_{N,j-1} - u_{N-1,j} = h q_{1,j}.$$

So: **diagonal 3 instead of 4, three -1 s instead of four, and the known flux times h on the right-hand side**. The missing fourth neighbour's role is played by the flux. In the code this is the `if i == interface_i` branch:

```

if i == interface_i:
    inward_i = n - 1 if side == "right" else 1
    neighbors = [(inward_i, j), (i, j-1), (i, j+1)] # only three
    diagonal = 3.0
else:
    neighbors = [(i-1, j), (i+1, j), (i, j-1), (i, j+1)]
    diagonal = 4.0

```

The matrices at $\Delta x = 1/3$

With $n = 3$ the unknowns are ordered row-major as

$$v_1 = (u_{1,1}^{(1)}, u_{2,1}^{(1)}, u_{3,1}^{(1)}, u_{1,2}^{(1)}, u_{2,2}^{(1)}, u_{3,2}^{(1)})^T,$$

and similarly for v_3 (whose interface column is $i = 0$) and v_2 (a 2×5 block of interior nodes). This gives $A_1, A_3 \in \mathbb{R}^{6 \times 6}$ and $A_2 \in \mathbb{R}^{10 \times 10}$:

```

A1 =
[[ 4. -1.  0. -1.  0.  0.]
 [-1.  4. -1.  0. -1.  0.]
 [ 0. -1.  3.  0.  0. -1.]
 [-1.  0.  0.  4. -1.  0.]
 [ 0. -1.  0. -1.  4. -1.]
 [ 0.  0. -1.  0. -1.  3.]]

A3 =
[[ 3. -1.  0. -1.  0.  0.]
 [-1.  4. -1.  0. -1.  0.]
 [ 0. -1.  4.  0.  0. -1.]
 [-1.  0.  0.  3. -1.  0.]
 [ 0. -1.  0. -1.  4. -1.]
 [ 0.  0. -1.  0. -1.  4.]]

A2 =
[[ 4. -1. -1.  0.  0.  0.  0.  0.  0.  0.]
 [-1.  4.  0. -1.  0.  0.  0.  0.  0.  0.]
 [-1.  0.  4. -1. -1.  0.  0.  0.  0.  0.]
 [ 0. -1. -1.  4.  0. -1.  0.  0.  0.  0.]
 [ 0.  0. -1.  0.  4. -1. -1.  0.  0.  0.]
 [ 0.  0.  0. -1. -1.  4.  0. -1.  0.  0.]
 [ 0.  0.  0.  0. -1.  0.  4. -1. -1.  0.]
 [ 0.  0.  0.  0.  0. -1. -1.  4.  0. -1.]
 [ 0.  0.  0.  0.  0.  0. -1.  0.  4. -1.]
 [ 0.  0.  0.  0.  0.  0.  0. -1. -1.  4.]]

```

Points to make about these matrices:

- **Where the 3s sit.** In A_1 they are in rows 3 and 6 — the *last* node of each grid row, because Ω_1 's interface is on its **right**. In A_3 they are in rows 1 and 4 — the *first* node of each row, because Ω_3 's interface is on its **left**. Same structure, mirrored.
- A_2 **has no 3s at all**, because the middle room has Dirichlet conditions on every edge.
- **Banded structure.** The -1 s appear next to the diagonal (left/right neighbours) and a fixed distance away (up/down neighbours), that distance being the number of unknowns per grid row. A_2 is 10×10 because its interior is 2 nodes wide and 5 tall.
- **Symmetric and diagonally dominant**, which is why a direct solve is stable and why the matrix is invertible.

7. Relaxation

Rather than adopting each newly computed answer outright, the iteration blends it with the previous one:

$$u^{k+1} \leftarrow \omega u_{\text{new}}^{k+1} + (1 - \omega) u^k, \quad \omega = 0.8.$$

```
def relax(old, new, omega):
    return omega * new + (1.0 - omega) * old
```

The intuition is a shower tap: nudge it rather than yanking it, or you overshoot and oscillate. In the Dirichlet–Neumann iteration the two sides of an interface tend to over-respond to each other, and damping the update controls that.

We measured the effect directly by re-running the 2-room case at $\Delta x = 1/20$ for a range of ω and fitting the geometric rate at which the interface update shrinks per iteration (smaller is better):

ω	0.2	0.4	0.55	0.60	0.65	0.70	0.80	0.90	1.00
rate / iteration	0.73	0.50	0.31	0.26	0.19	0.28	0.46	0.64	0.82

A genuine finding for Task 4

The prescribed $\omega = 0.8$ is **not** optimal for this problem. The best value is near $\omega \approx 0.65$, which contracts the error by a factor 0.19 per iteration against 0.46 at $\omega = 0.8$ — more than twice as fast. It reaches an interface update below 10^{-6} °C in 11 iterations instead of more than 14.

Note also that $\omega = 1$ (no relaxation at all) does **not** diverge here: it still converges, at rate 0.82, just slowly. That is a useful correction to the common assumption that the iteration falls apart without damping — on this geometry it does not, it merely becomes inefficient.

8. How the code is organised

Three files, with a deliberate separation:

File	Contains	Why
<code>solver.py</code>	All the numerics. No MPI, no plotting.	Testable and reusable; both drivers and the notebook import from it.
<code>mpi_driver.py</code> <code>mpi_driver_half.py</code>	Only the process orchestration: who solves what, who sends what to whom.	A Jupyter kernel is a single process, so the parallel part cannot live in the notebook.
<code>project01.ipynb</code>	The report: derivation, experiment launches, results, plots.	The assignment artifact.

The numerical implementation is deliberately *not* duplicated between the notebook and the drivers — both import the same functions.

The functions in `solver.py`

Function	What it does
<code>interior_index</code>	Flattens a grid coordinate (i, j) to a vector index.
<code>build_matrix</code>	Turns collected <code>(row, column, value)</code> triplets into a dense array or a sparse matrix.
<code>solve_system</code>	Dispatches to <code>scipy.linalg.solve</code> or <code>scipy.sparse.linalg.spsolve</code> depending on the matrix format.
<code>dirichlet_matrix</code>	Builds A_2 : the ordinary five-point matrix for a room with Dirichlet data all round.
<code>outer_unknowns</code>	Lists which nodes of an outer room are unknown — crucially <i>including</i> the interface column.
<code>neumann_matrix</code>	Builds A_1, A_3, A_4 , with the 3-diagonal flux-balance rows on the interface.
<code>center_boundary</code> , <code>outer_boundary</code> , <code>center_boundary_half</code> , <code>small_room_boundary</code>	Paint a room's boundary values onto its grid: heaters, window, ordinary walls, and the current interface guess.
<code>dirichlet_b</code> , <code>outer_b</code>	Build the right-hand side, moving known wall values across and inserting the $h q$ flux terms.
<code>solve_center</code> , <code>solve_outer</code>	Assemble b , solve, and write the answer back into the two-dimensional grid.
<code>interface_fluxes</code> , <code>interface_fluxes_half</code>	Measure $\partial u / \partial n$ on each interface from the middle room's solution.
<code>relax</code>	The ω blend.
<code>assemble</code> , <code>assemble_apartment</code> , <code>assemble_apartment_half</code>	Glue the room grids into one array for plotting, averaging shared interface columns and leaving NaN where there is no room.

The one subtlety in `outer_unknowns`

```
def outer_unknowns(n, side):
    x_nodes = range(1, n + 1) if side == "right" else range(n)
    return [(i, j) for j in range(1, n) for i in x_nodes]
```

For a room whose interface is on its **right**, the columns run `1..n` — column n is the interface and *is* included. For a room whose interface is on its **left**, they run `0..n-1` — column 0 is the interface, again included. Column 0 (respectively n) is excluded on the other side because that is the exterior wall, which is known. Getting this list right is what makes the sizes of A_1 and A_3 come out as $n \cdot (n - 1)$ rather than $(n - 1)^2$.

Assembling the picture

```
def assemble(placements, shape):
    total = np.zeros(shape)
    count = np.zeros(shape)
    for values, row0, column0 in placements:
        rows = slice(row0, row0 + values.shape[0])
        columns = slice(column0, column0 + values.shape[1])
        total[rows, columns] += values
        count[rows, columns] += 1.0
    apartment = np.full(shape, np.nan)
    apartment[count > 0] = total[count > 0] / count[count > 0]
    return apartment
```

Rooms are pasted into one big array at given offsets. `count` records how many rooms covered each cell, so shared interface columns get averaged rather than double-counted, and cells no room covered stay `NaN` — which is what leaves the empty corners of the L-shape blank in the plots.

9. The MPI layer

MPI (Message Passing Interface) runs several copies of the same program as separate processes. Each copy gets a **rank**, an integer identifying it, and the program branches on that rank to decide what that copy is responsible for. Processes do not share memory; they exchange data by explicit messages.

Rank	Room	Condition on its interface
0	Ω_2 (middle)	Dirichlet — and it owns the convergence history
1	Ω_1 (left)	Neumann
2	Ω_3 (right)	Neumann

Rank 0's loop, which is the heart of the program:

```

for _ in range(iterations):
    boundary = center_boundary(n, gamma1, gamma2, wall, heater, window)
    raw_u2 = solve_center(A2, boundary)

    # Compute the neumann conditions directly and send them to rank1 and rank2
    flux1, flux3 = interface_fluxes(raw_u2, h)
    comm.send(flux1, dest=1, tag=11)
    comm.send(flux3, dest=2, tag=21)

    # Receive the new dirichlet condition
    new_gamma1 = comm.recv(source=1, tag=12)
    new_gamma2 = comm.recv(source=2, tag=22)

    change = max(np.max(np.abs(new_gamma1 - gamma1)),
                  np.max(np.abs(new_gamma2 - gamma2)))
    history.append(change)
    gamma1, gamma2 = new_gamma1, new_gamma2
    u2 = relax(u2, raw_u2, omega)

```

and the matching outer-room loop, here rank 1:

```

A1 = neumann_matrix(n, "right")
boundary = outer_boundary(n, "right", wall, heater, window)
comm.send(u1[1:-1, -1], dest=0, tag=10) # initial interface temperature

for _ in range(iterations):
    flux1 = comm.recv(source=0, tag=11)
    raw_u1 = solve_outer(A1, boundary, flux1, h, "right")
    u1 = relax(u1, raw_u1, omega)
    comm.send(u1[1:-1, -1], dest=0, tag=12) # hand the temperature back

```

Things worth pointing out:

- The **matrix is built once, outside the loop**. It never changes; only b changes from sweep to sweep. This matters for cost.
- The `tag` numbers are just labels. MPI matches a message by (source, tag), so distinct tags keep the flux messages and the temperature messages from being confused.
- `u1[1:-1, -1]` is the right-hand column of Ω_1 minus its two end points — the interior interface nodes, which is exactly the data Ω_2 needs.
- **Ranks 1 and 2 work simultaneously**. They do not depend on each other, only on rank 0. That is the parallelism in the method. At $\Delta x = 1/20$ each room's solve is so cheap that the demonstration is structural rather than a speed win; the structure is what would pay off on a large mesh.
- **Where ω actually acts**. Rank 0 relaxes its own `u2`, but it feeds the *un-relaxed* `raw_u2` into the flux computation. So the damping that steers the iteration is the one applied on ranks 1 and 2 to the interface temperatures they send back — which is exactly where the algorithm wants it.

Why the notebook shells out

```

command = ["mpiexec", "-n", "3", sys.executable, "mpi_driver.py", ...]
run = subprocess.run(command, capture_output=True, text=True, check=True)
return np.load(output)

```

A Jupyter kernel is a *single* process, so you cannot create MPI ranks inside it. The notebook therefore launches the parallel job as an external command with `mpiexec -n 3`, and the driver writes its results to an `.npz` file

which the notebook loads back. This is the standard way to drive MPI from a notebook.

10. Results for Project 1 (Tasks 2, 3, 4)

Task 2 — is the heating adequate?

The handout gives no threshold, so we state one: **18 °C**, measured at interior nodes only (a 40 °C radiator surface is not "room temperature", so wall nodes are excluded).

Quantity	Value
Interior minimum	5.96 °C
Interior mean	19.89 °C
Interior nodes below 18 °C	46.4 %
Interface update at the last iteration	0.012 °C

Answer: no. The mean is a comfortable 19.9 °C, but nearly half the flat is below 18 °C and the coldest corner is barely above 6 °C. The average flatters the flat; the distribution is what matters.

Task 3 — the temperature distribution

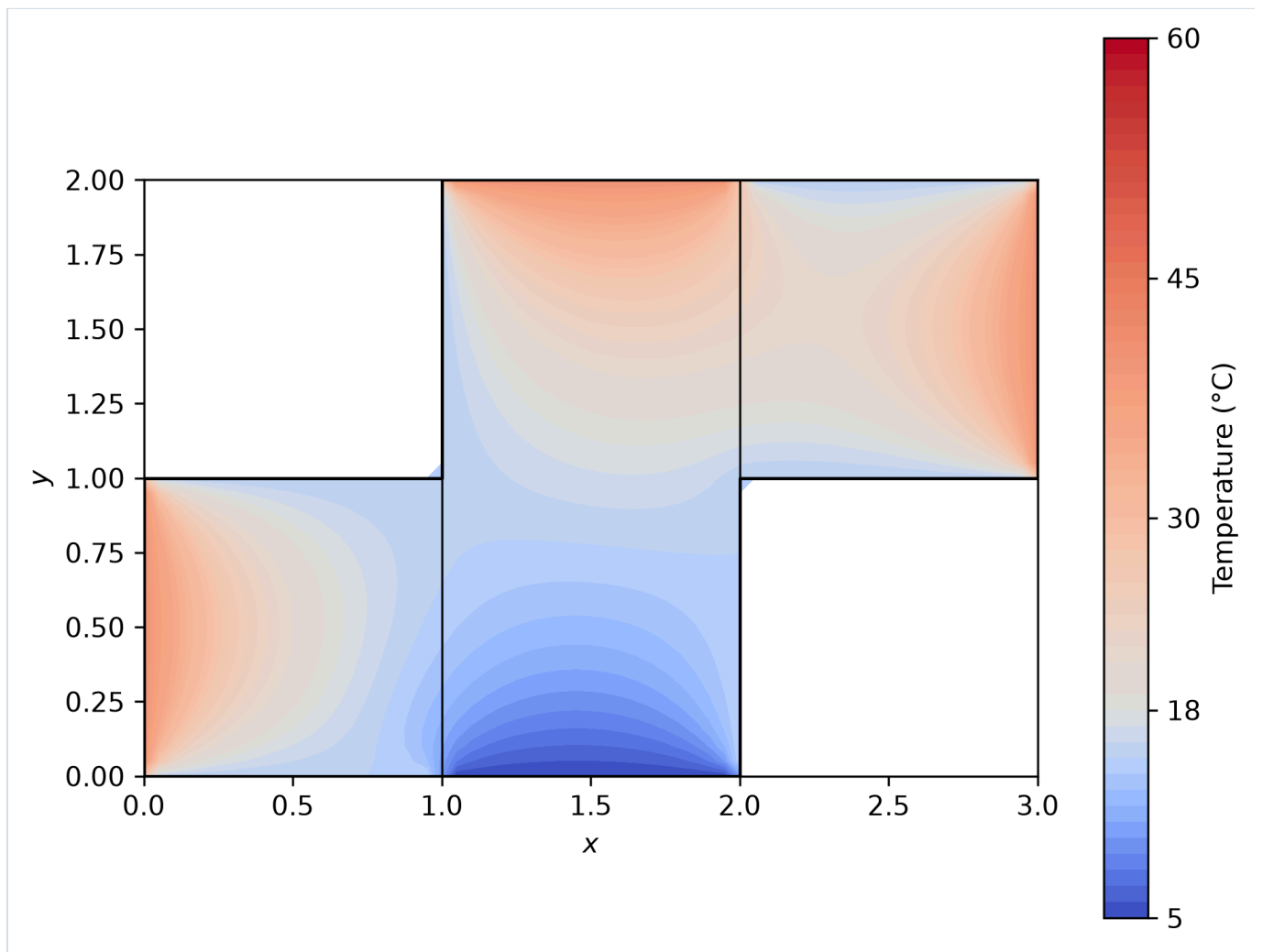


Figure 2. Steady temperature at $\Delta x = 1/20$, $\omega = 0.8$, ten iterations. The colour scale is deliberately centred at 18 °C, so **blue means below the comfort threshold and red above it** — you can read adequacy straight off the colours. The cold pocket sits over the window along the bottom of Ω_2 ; the three heater walls are the red bands.

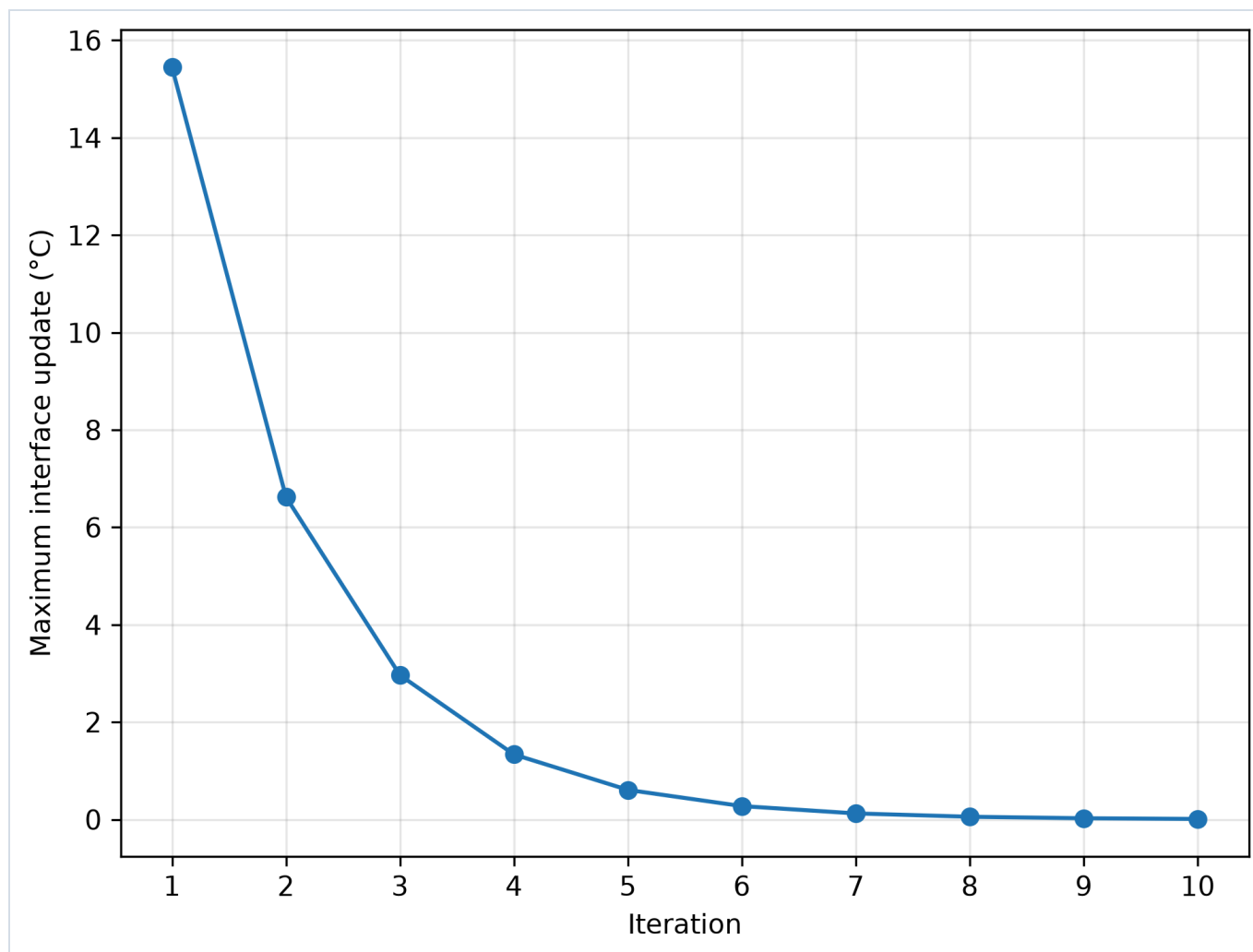


Figure 3. The largest change in interface temperature at each sweep. It falls from about 15 °C to 0.012 °C in ten iterations, which is the evidence that the handshake has actually converged and the three rooms agree.

Task 4 — varying the heater temperature

Heater	Minimum	Mean	Below 18 °C
40 °C	5.96 °C	19.89 °C	46.4 %
50 °C	6.00 °C	22.26 °C	40.1 %
60 °C	6.04 °C	24.63 °C	35.5 %

The most interesting result in Project 1

Raising the heaters by 20 °C lifts the *mean* by almost 5 °C but moves the *minimum* by 0.08 °C — essentially nothing. The coldest nodes are the ones directly adjacent to the fixed 5 °C window. Since every node is the average of its neighbours, a node one step from a 5 °C boundary is dominated by that boundary, and no amount of heat from across the flat can rescue it. **The window sets the minimum, not the heating.** Buy better glazing, not a bigger radiator.

11. Project 1a: the 2.5-room apartment

The add-on replaces the flat with a very similar one carrying a small fourth room, and asks whether the existing code extends or has to be rewritten.

Reading the geometry

Room	Extent	Interface
Ω_1	$[0, 1] \times [0, 1]$	Γ_1 on its right edge
Ω_2	$[1, 2] \times [0, 2]$	Dirichlet on $\Gamma_1, \Gamma_2, \Gamma_3$
Ω_3	$[2, 3] \times [1, 2]$	Γ_2 on its left edge
Ω_4	$[2, \frac{5}{2}] \times [\frac{1}{2}, 1]$	Γ_3 on its left edge

Ω_4 is $\frac{1}{2} \times \frac{1}{2}$, one quarter of the area of Ω_1 , as the handout specifies. It touches Ω_2 **only** through Γ_3 : its ceiling separates it from Ω_3 but is not drawn as an interface in the figure, so it is an ordinary wall. That is what makes it the "half room" of a 2.5-room flat — reachable only from the middle room. Its floor is a heater.

The right edge of Ω_2 is therefore now three segments: Γ_2 above $y = 1$, Γ_3 between $y = \frac{1}{2}$ and $y = 1$, and an ordinary 15 °C wall below $y = \frac{1}{2}$.

A new constraint on the mesh

Ω_4 has side $\frac{1}{2}$, so it only lands on the grid when $n = 1/\Delta x$ is **even**. The $\Delta x = 1/3$ used for Task 1 of Project 1 is therefore *not admissible* for this layout, and $\Delta x = 1/4$ takes its place. This is an easy question to be caught out by in a presentation.

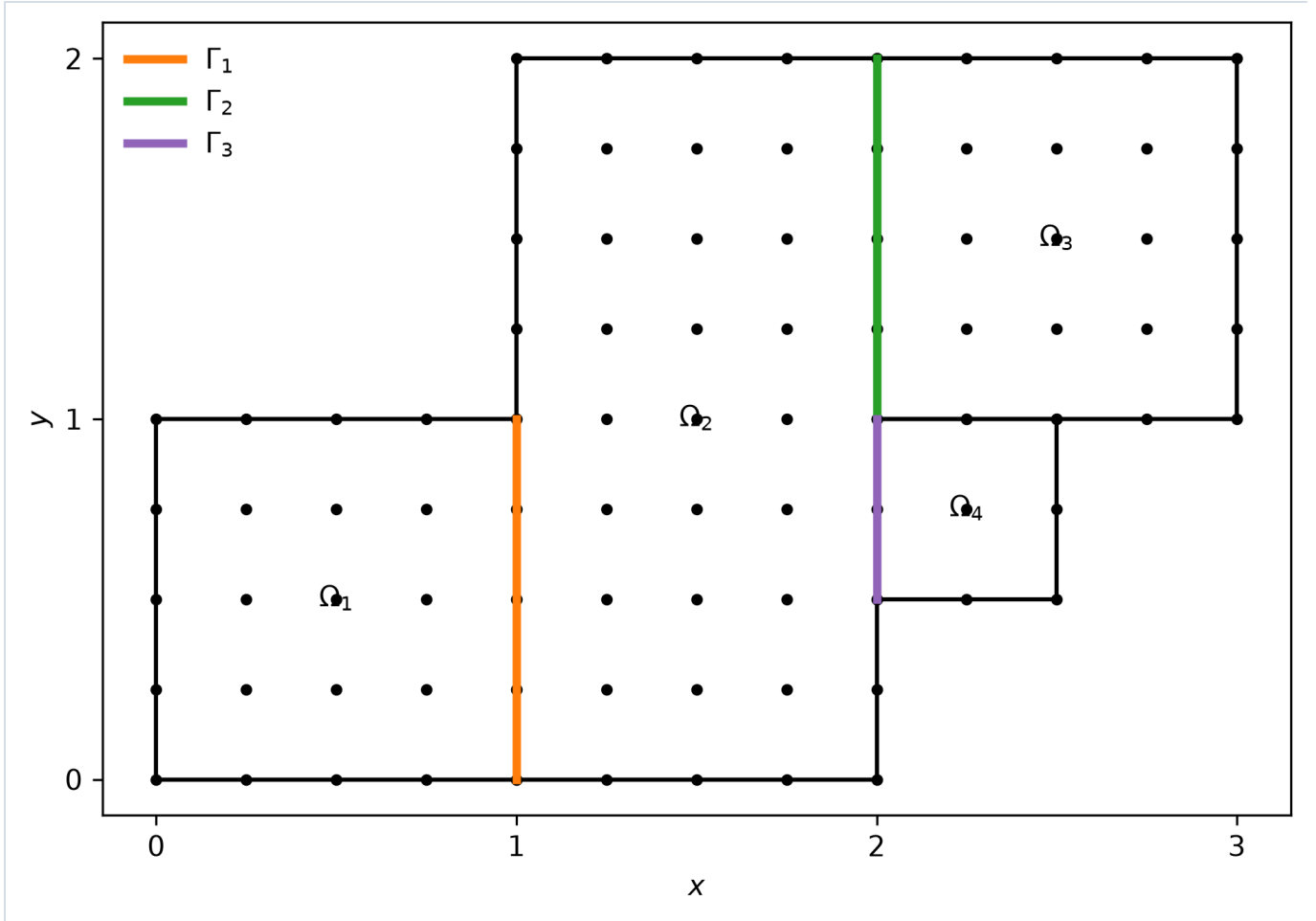


Figure 4. The 2.5-room flat at $\Delta x = 1/4$, with the three interfaces marked. Ω_4 hangs off the right wall of Ω_2 , below Ω_3 .

The matrices at $\Delta x = 1/4$

With $n = 4$ and $m = n/2 = 2$: $A_1, A_3 \in \mathbb{R}^{12 \times 12}$ (three rows of four nodes), $A_2 \in \mathbb{R}^{21 \times 21}$ (three columns of seven), and $A_4 \in \mathbb{R}^{2 \times 2}$ — a single row of two nodes:

```
A4 =
[[ 3. -1.]
 [-1.  4.]]
```

Small as it is, A_4 shows the whole pattern in miniature: the first node is on the interface and gets the 3, the second is interior and gets the 4.

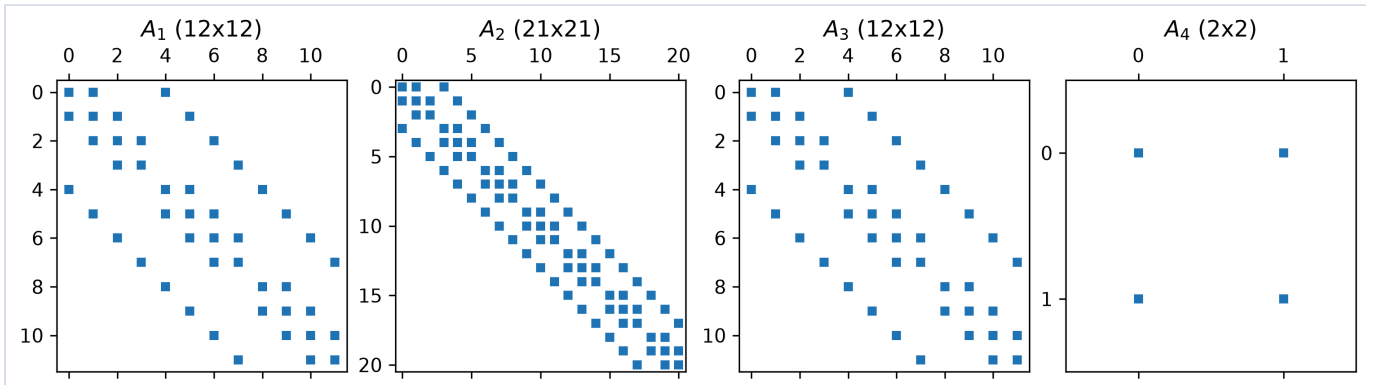


Figure 5. Sparsity patterns. The banded structure is identical in kind across the four rooms; only the size and the position of the interface rows differ. This picture is also the argument for sparse storage in Section 13.

Results

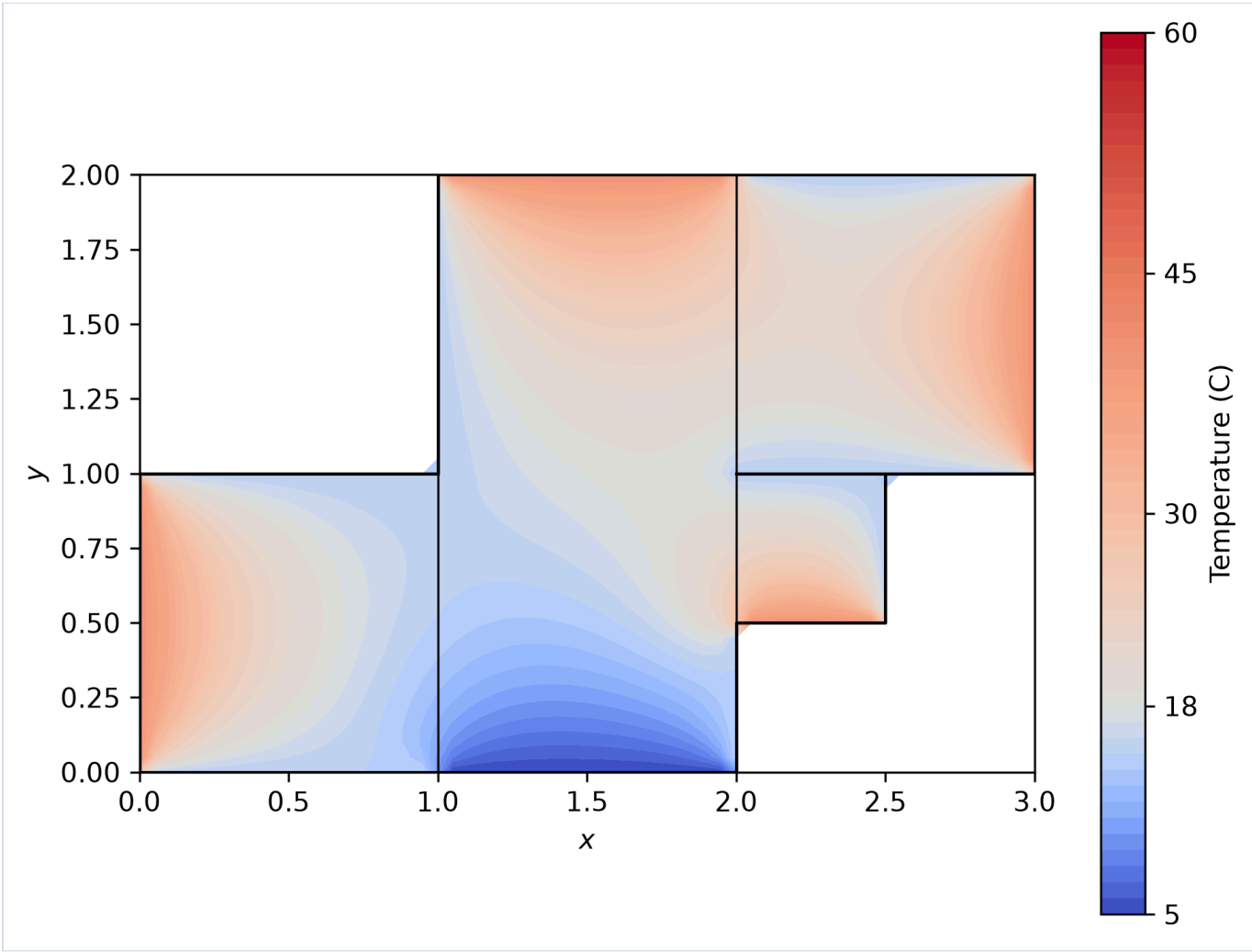


Figure 6. The 2.5-room flat at $\Delta x = 1/20$. Ω_4 's floor heater is the warm patch bottom right; the cold pocket at the window is unchanged.

Layout	Minimum	Mean	Below 18 °C
2 rooms	5.96 °C	19.89 °C	46.4 %
2.5 rooms	6.11 °C	20.54 °C	39.8 %

The extra heated half-room helps but does not fix the flat: the mean rises by 0.65 °C and the cold fraction drops from 46 % to 40 %, both because Ω_4 is itself warm and because it feeds heat back into the lower right of Ω_2 through Γ_3 . But the minimum moves by 0.15 °C, for exactly the reason given in Task 4: the coldest nodes are still pinned against the window. **The verdict is unchanged — the heating is not adequate.**

Verifying that the coupling actually worked

A useful slide, because it is the check that the method did what it claims. After ten iterations the temperature computed on the two sides of each interface agrees to:

Interface	Γ_1	Γ_2	Γ_3
Largest disagreement between the two rooms	0.0045 °C	0.0081 °C	0.0006 °C

The whole field also stays within $[5, 40]$ °C, which it must: with no interior heat source, the maximum principle says the extremes have to occur on the boundary.

12. Was the code extendible?

This is the actual question Project 1a asks, and it deserves a slide of its own. Sorting the Project 1 code by what happened to it:

Reused completely unchanged

- `interior_index`, `dirichlet_b`, `outer_b` — the stencil and right-hand side assembly are layout-independent.
- `dirichlet_matrix` — A_2 does not depend on *which* boundary nodes are walls and which are interfaces, only on which nodes are unknown. The middle-room matrix is built by exactly the same call even though its boundary now carries three interface segments.
- `neumann_matrix` — Ω_4 is still a square room whose interface runs along one full edge, so `neumann_matrix(m, "left")` produces A_4 with no change at all.
- `relax`, and the overall rank structure of the driver.

Had to be generalised — three hidden assumptions

1. `solve_center` and `solve_outer` each built their own boundary grid internally, which hardcoded where the heaters and walls are. They now take a *prepared* boundary array, so a room with different walls is just a different array.
2. `solve_outer` derived the mesh width as `1.0 / n` from the size of the room's own matrix. See the box below.
3. `assemble_apartment` had the three room positions written out as literal array slices. It became `assemble(placements, shape)`, taking a list of grids with offsets, so each layout is one short list.

A real bug the extension exposed

Computing the mesh width as $1/n$ from a room's own matrix size is correct only while *every room has side*

1. Ω_4 has side $\frac{1}{2}$ covered by $m = n/2$ intervals, so its mesh width is $h = \frac{1/2}{n/2} = 1/n$, **not** $1/m = 2/n$. Left alone, the code would have used a flux term $h q$ twice too large at Γ_3 and produced a plausible-looking but wrong answer — no crash, no warning. The mesh width is now an explicit argument. This is a good example of an assumption that is invisible until the geometry changes.

Genuinely new — only the geometry description

- `center_boundary_half` — the right edge of Ω_2 split into two interface segments and a wall.
- `small_room_boundary` — Ω_4 's walls, heater along the floor.
- `interface_fluxes_half` — one extra flux slice.
- `mpi_driver_half.py` — four ranks instead of three.

Conclusion for the presentation: the numerics carried over completely; the geometry did not. The code *was* extendible, but only after pulling three hardcoded layout assumptions out of functions that had no business knowing about them. Project 1 still reproduces bit-for-bit identical results after the refactoring, which is how we know nothing was broken in the process.

13. Task 2 of Project 1a: computing at $\Delta x = 1/100$

Why the Project 1 approach stops working

At $\Delta x = 1/100$ the middle room has $99 \times 199 = 19\,701$ unknowns. `scipy.linalg.solve` needs a *dense* matrix, so A_2 alone would occupy

$$19\,701^2 \times 8 \text{ bytes} \approx 2.9 \text{ GB},$$

before the LU factorisation allocates a working copy of its own.

Room	Unknowns	Dense	Sparse
Ω_1 / Ω_3	9 900	0.73 GB	0.57 MB
Ω_2	19 701	2.89 GB	1.13 MB
Ω_4	2 450	0.04 GB	0.14 MB

The method is fine; the *storage* is hopeless. Each row of a five-point stencil has at most five non-zero entries, so A_2 holds about 10^5 meaningful numbers out of 3.9×10^8 array slots — it is more than 99.9 % zeros.

The fix

The matrix builders now collect their entries as `(row, column, value)` triplets and hand them to either a dense array or a `scipy.sparse` matrix:

```
def build_matrix(rows, columns, values, size, sparse):
    if sparse:
        return coo_matrix((values, (rows, columns)), shape=(size, size)).tocsr()
    A = np.zeros((size, size))
    A[rows, columns] = values
    return A

def solve_system(A, b):
    return spsolve(A, b) if issparse(A) else solve(A, b)
```

The stencil logic is untouched. This is a change of storage, not of method — a good thing to be able to say clearly. The two paths were checked against each other at $\Delta x = 1/20$ and agree to 1.07×10^{-13} , i.e. to round-off.

Results at $\Delta x = 1/100$

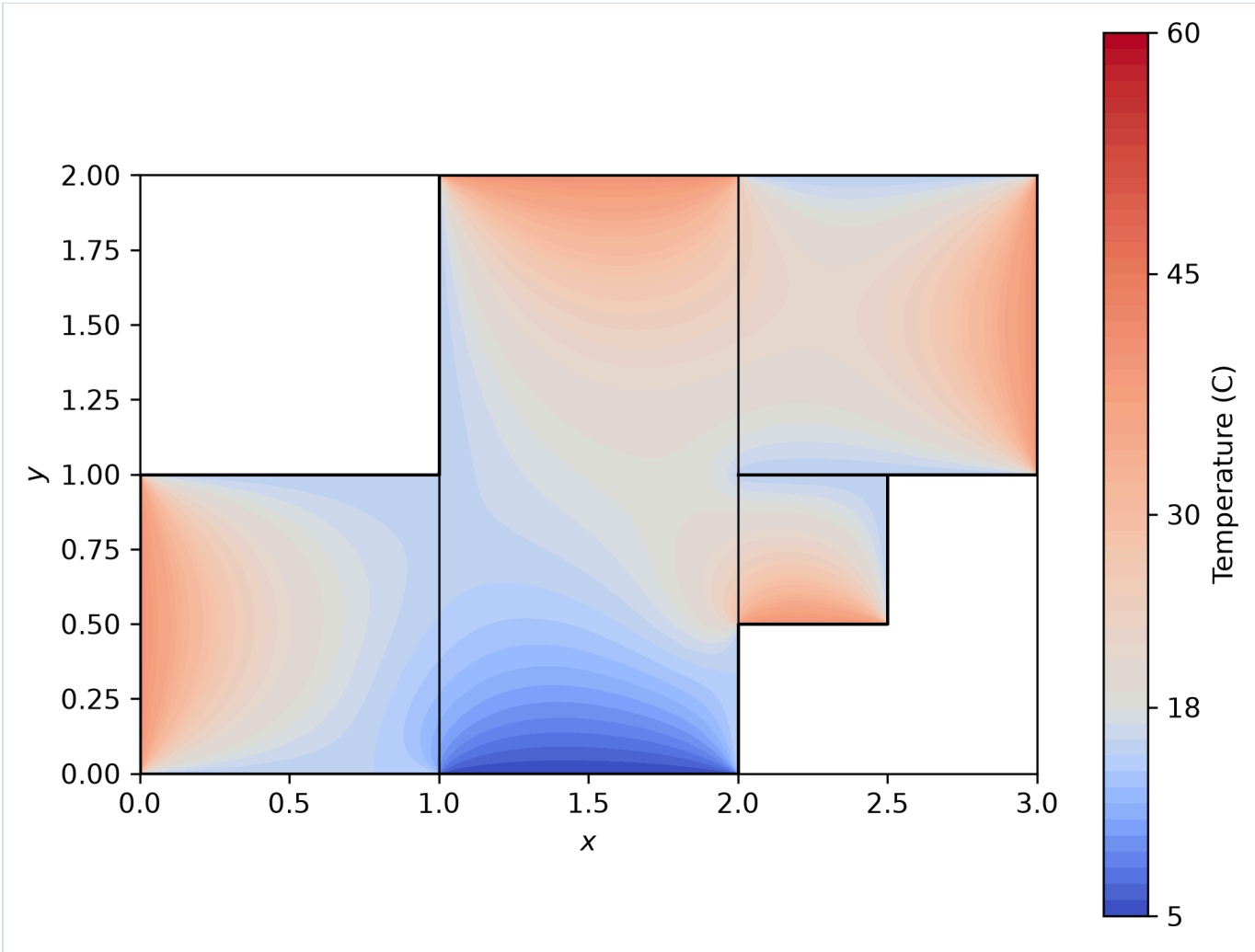


Figure 7. The 2.5-room flat at $\Delta x = 1/100$: the same picture as Figure 6, better resolved. The run takes about 5 seconds for 30 iterations.

Mesh	Minimum	Mean	Below 18 °C	Unknowns (total)
$\Delta x = 1/20$	6.11 °C	20.54 °C	39.8 %	1 604
$\Delta x = 1/100$	5.22 °C	20.65 °C	40.4 %	42 004

The mean shifts by 0.11 °C, which is the discretisation error of the coarse mesh. The minimum drops more (6.11 to 5.22 °C) because the finer grid places nodes physically closer to the cold window corner, where the solution changes fastest.

Convergence depends on the mesh

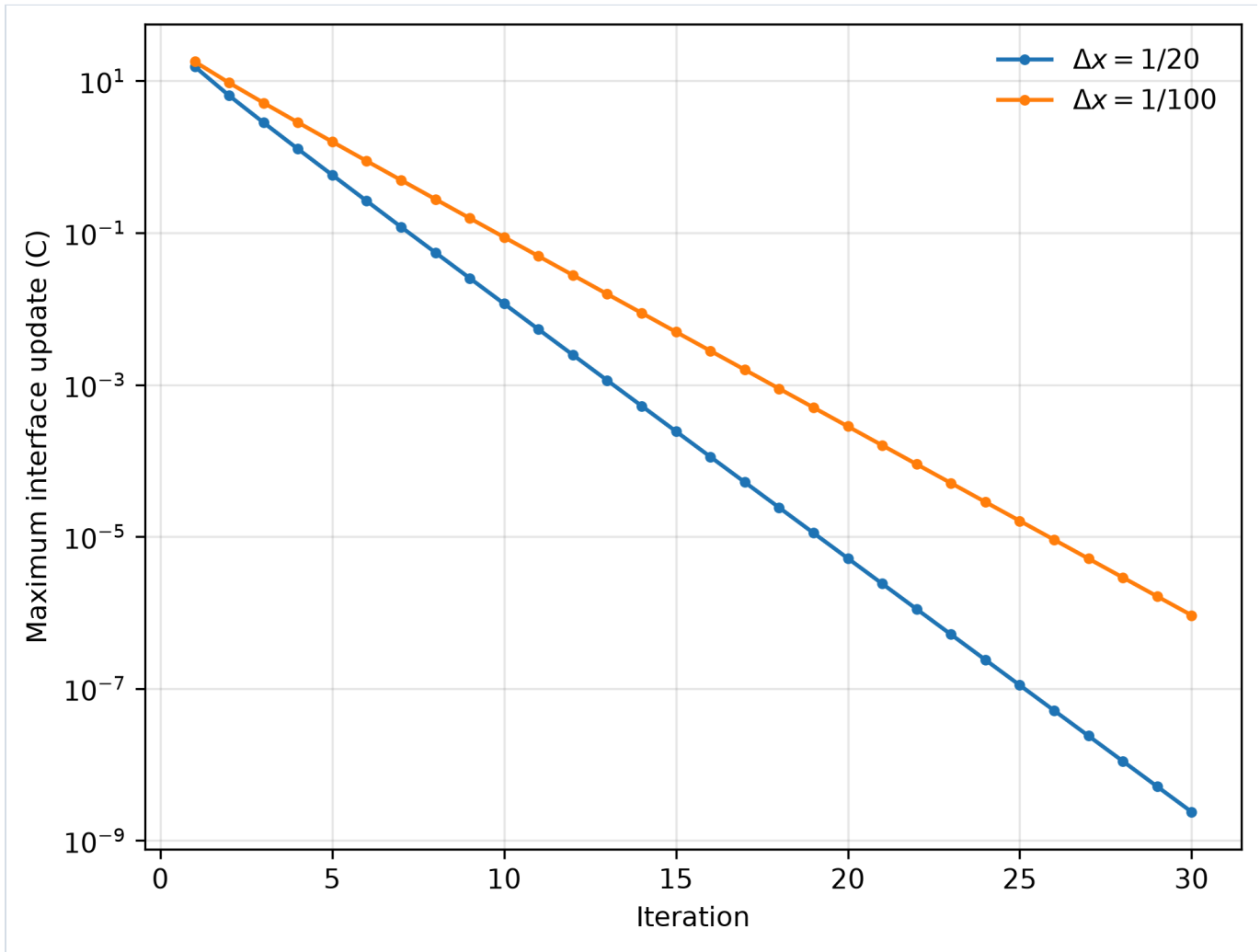


Figure 8. Interface update per iteration on a logarithmic axis. Both meshes contract geometrically — a straight line on this axis — but the finer mesh contracts more slowly.

Iterations needed to reach	$10^{-1} \text{ }^{\circ}\text{C}$	$10^{-2} \text{ }^{\circ}\text{C}$	$10^{-3} \text{ }^{\circ}\text{C}$	rate / iteration
$\Delta x = 1/20$	8	11	14	0.46
$\Delta x = 1/100$	10	14	18	0.56

Refining the mesh costs more per iteration *and* a few more iterations. But the increase is mild — 18 against 14 — and the count does not blow up. That mild mesh-dependence is the property that makes the Dirichlet–Neumann method worth using. It also means **ten iterations are not enough at $\Delta x = 1/100$** , which is why that case is run for 30.

14. Summary of all results

Case	Min	Mean	< 18 °C	Note
2 rooms, $\Delta x = 1/20$, 10 iter	5.96	19.89	46.4 %	Project 1 baseline
2 rooms, heater 50 °C	6.00	22.26	40.1 %	Task 4
2 rooms, heater 60 °C	6.04	24.63	35.5 %	Task 4
2.5 rooms, $\Delta x = 1/20$, 10 iter	6.11	20.54	39.8 %	Project 1a Task 1
2.5 rooms, $\Delta x = 1/100$, 30 iter	5.22	20.65	40.4 %	Project 1a Task 2, sparse, ~5 s

All temperatures in °C, measured at interior nodes only, with the comfort threshold taken as 18 °C.

How to reproduce

```
# environment
uv sync

# the notebook (VS Code: select the .venv kernel, then Run All)
project1/project01.ipynb

# a single run directly
cd project1
mpiexec -n 3 python mpi_driver.py      --n 20  --omega 0.8 --iterations 10 \
    --wall 15 --heater 40 --window 5 --output out.npz
mpiexec -n 4 python mpi_driver_half.py --n 100 --omega 0.8 --iterations 30 \
    --wall 15 --heater 40 --window 5 --sparse --output fine.npz
```

Practical trap

If `solver.py` changes while a Jupyter kernel is running, the kernel keeps the *old* module — Python caches imports and re-running an `import` statement does not re-read the file. The symptom is an `ImportError` for a function you can see in the file. The fix is **Restart Kernel and Run All**, not re-running the failing cell.

15. Presentation crib sheet

The questions most likely to come up, with short answers.

Why is the diagonal 3 instead of 4 on some rows?

Those rows belong to interface nodes of an outer room. They sit on the room's edge, so only three of the four neighbours exist. The missing fourth direction is supplied by the prescribed heat flux, which is a known quantity and therefore goes to the right-hand side as $h q$.

Why is the interface an unknown in the outer rooms but not in the middle one?

Because of who is told what. The middle room is *given* the interface temperature (Dirichlet), so those nodes are known and move to b . The outer rooms are given the flux, not the temperature, so their interface temperature is

precisely what they solve for.

Why Dirichlet on the middle room and Neumann on the outer ones, rather than the reverse?

The assignment prescribes it. The intuition is that the middle room is the hub, touching both (or all three) neighbours: giving it the temperatures from all of them makes its problem well-posed and lets it hand back a consistent set of fluxes in one solve. The choice does affect how fast the iteration converges.

Where did the mesh width h go in the matrix?

Multiplying the discrete Laplace equation by $-h^2$ cancels it. It reappears only in the Neumann rows, as the $h q$ term on the right-hand side.

Is the solution inside a room approximate?

The *discretisation* is approximate to $O(h^2)$, but the linear system is solved exactly by a direct method. There is no iteration inside a room — only between rooms.

What exactly is being iterated, then?

The interface temperatures. The Dirichlet–Neumann iteration is a fixed-point iteration on those few numbers; everything else is a consequence of them. That is also why the convergence history plots the largest change in interface temperature.

How do you know it converged?

Three checks. The interface update falls geometrically to about 10^{-2} °C (Figure 3). The temperature computed from the two sides of each interface agrees to under 0.01 °C. And the whole field stays inside $[5, 40]$ °C, as the maximum principle requires.

What does ω do, and what happens at $\omega = 1$?

It damps the update, blending the new interface values with the old. At $\omega = 1$ there is no damping; on this geometry it still converges, but slowly (rate 0.82 per iteration against 0.46 at $\omega = 0.8$). We measured the optimum at about $\omega \approx 0.65$, rate 0.19 — more than twice as fast as the prescribed value.

Why doesn't a hotter radiator fix the cold corner?

Because every node is the average of its neighbours, a node one step away from a fixed 5 °C boundary is dominated by that boundary. Going from 40 to 60 °C moves the minimum by 0.08 °C. The window sets the minimum.

Why does the fine mesh have a lower minimum?

Not because it is colder, but because it resolves better: the finer grid puts nodes physically closer to the 5 °C window, where the solution varies fastest, so it captures a more extreme value the coarse grid stepped over.

Why sparse matrices, and does it change the answer?

At $\Delta x = 1/100$ a dense A_2 needs 2.9 GB against about 1 MB sparse, because the matrix is over 99.9 % zeros. It is a change of storage only — the stencil is identical, and dense and sparse agree to 10^{-13} .

What does MPI actually buy you here?

The two (or three) outer rooms are independent of each other and solve concurrently; only the middle room is a synchronisation point. At $\Delta x = 1/20$ each solve is far too cheap for this to show up as wall-clock speedup — the value is that the structure is the one that scales, and the exercise is about the method.

What would you do differently?

Three honest answers. Use $\omega \approx 0.65$ instead of 0.8. Describe the layout as data — a list of rooms and interfaces — from the start, rather than hardcoding slices, so a new floor plan needs no code changes at all. And add a convergence tolerance as a stopping criterion instead of a fixed iteration count, since the right number of sweeps depends on the mesh.